

Task 2 - Web Application Security Basics Report

Target: DVWA (Damn Vulnerable Web Application)

Tools Used: Burp Suite Community Edition

Vulnerabilities Tested: SQL Injection (SQLi), Reflected Cross-Site Scripting (XSS)

1. Introduction

This assessment was conducted on the DVWA application hosted on `localhost` to evaluate common web security vulnerabilities. The focus was on identifying SQL Injection and Reflected XSS vulnerabilities, demonstrating their impact, and recommending mitigation strategies.

Burp Suite was used as a proxy to capture, inspect, and modify HTTP requests and responses, providing visibility into how the application handles user input.

2. Test Environment and Tools

- **Application:** DVWA v1.10 (Development Mode)
- **Browser:** Chromium-based, configured to route traffic through Burp Suite proxy
- **Database:** MySQL/MariaDB (default DVWA configuration)
- **Security Level:** Low
- **Proxy Tool:** Burp Suite Community Edition

Burp Suite Configuration:

- Proxy interception enabled to capture requests
- Intercept mode used to modify input parameters
- Passive crawl used to build a site map

3. Methodology

The assessment methodology included:

1. Configuring Burp Proxy and browser to route traffic through it.
2. Enabling **Intercept** to capture and modify HTTP requests.
3. Performing SQL Injection tests on the `id` parameter of the SQLi page.
4. Performing Reflected XSS tests on the input fields of the XSS page.
5. Observing server responses to confirm vulnerabilities.
6. Documenting all findings with screenshots.
7. Recommending mitigation strategies based on observed vulnerabilities.

4.1 SQL Injection (SQLi)

4.1.1 Initial Testing and Error Observation

The initial SQL injection attempt involved modifying the `id` parameter in the intercepted request:

```
GET /vulnerabilities/sql/?id=1' OR 1=1--&Submit=Submit HTTP/1.1
```

SQL Injection 1st Method:



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

Vulnerability: SQL Injection

User ID:

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-okw/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_Injection
- <http://bobby-tables.com/>

This payload resulted in a **database syntax error**, indicating that the query structure was broken.

SQL Injection Error:

You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near ''' at line 1

Analysis:

- The error message revealed that the injected query was not properly terminated.
- This confirmed that user input was being directly inserted into an SQL query without sanitization.
- The presence of detailed database error messages provided useful feedback for refining the payload.

4.1.2 Refinement Using MariaDB Comment Syntax

Let's check for the MariaDB Comment Syntax:

SQL Syntax check for MariaDB:

1. From a hash sign (#) to the end of a line:

```
SELECT * FROM users;
```

```
# This is a comment
```

In MariaDB, the # symbol is used to comment out the rest of the SQL statement.

To resolve the syntax issue, a comment sequence was added to terminate the remaining part of the query:

```
1' OR 1=1#
```

SQL Injection 2nd Method:



Vulnerability: SQL Injection

User ID:

4.1.3 Successful Exploitation

After applying the corrected payload, the application returned multiple user records instead of a single entry.

SQL Injection Output:



Vulnerability: SQL Injection

User ID:

```
ID: 1' OR 1=1#  
First name: admin  
Surname: admin  
  
ID: 1' OR 1=1#  
First name: Gordon  
Surname: Brown  
  
ID: 1' OR 1=1#  
First name: Hack  
Surname: Me  
  
ID: 1' OR 1=1#  
First name: Pablo  
Surname: Picasso  
  
ID: 1' OR 1=1#  
First name: Bob  
Surname: Smith
```

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet>
- https://www.owasp.org/index.php/SQL_Injection
- <http://bobby-tables.com/>

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Observed Output:

- Multiple usernames (admin, Gordon, Hack, Pablo, Bob) were displayed.
- The query condition was effectively bypassed.

4.1.4 Explanation of the Attack

The original backend query is likely structured as:

```
SELECT * FROM users WHERE id = '$id';
```

After Injecting the payload:

```
SELECT * FROM users WHERE id = '1' OR 1=1#';
```

- '1' closes the original condition
- OR 1=1 forces the query to return all rows
- # comments out the trailing '

4.2 Reflected Cross-Site Scripting (XSS)

4.2.1 Initial Injection Attempt

- Entered script payload:

```
<script>alert(2)</script>
```

- Inserted into the "What's your name?" field.

XSS (Cross Site Scripting)



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello

More Information

- [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

Analysis:4.2.2 Successful XSS Execution

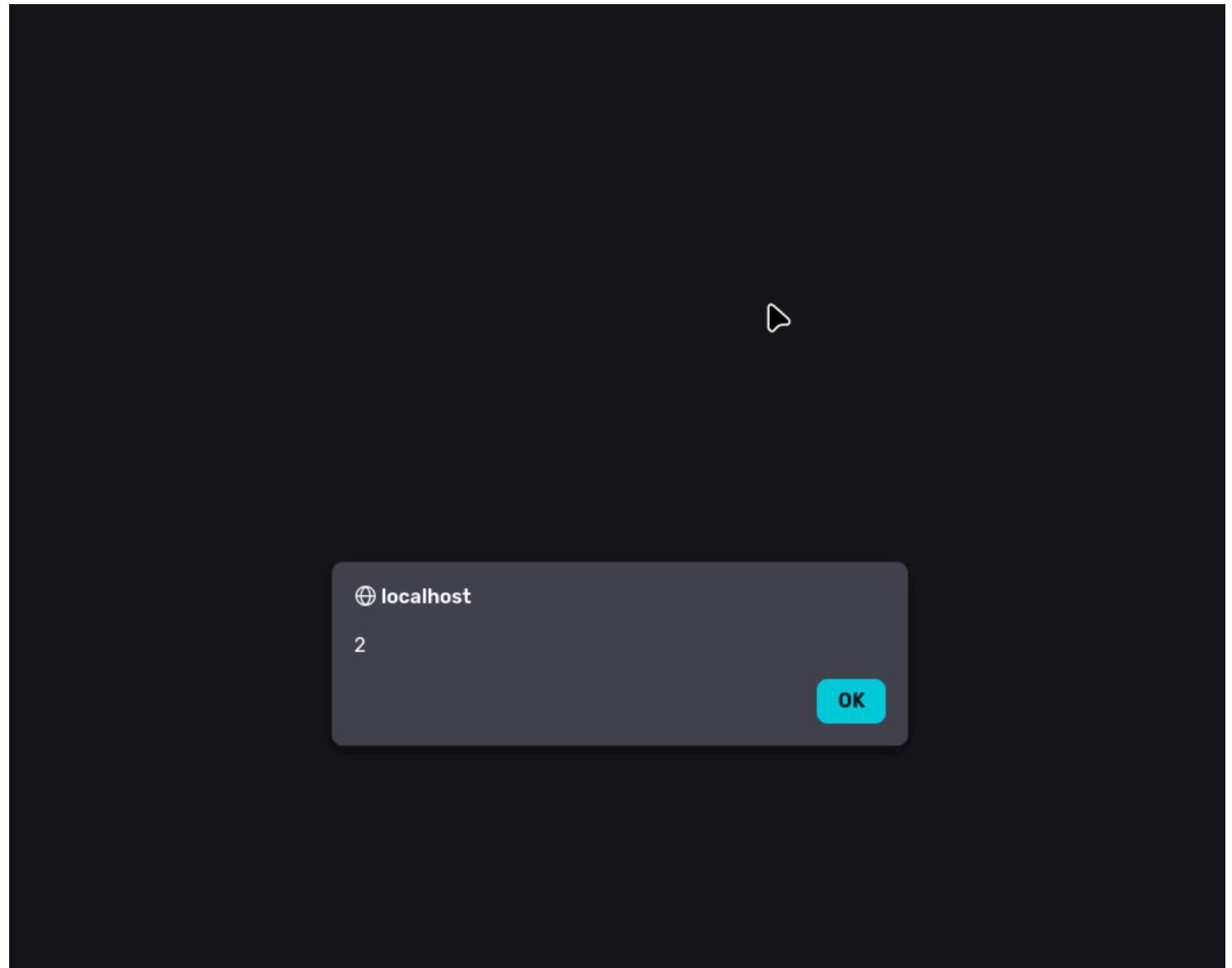
Input and output confirmed execution after adjusting context:

- The application reflected input but may have initially filtered/encoded it.

4.2.2 Execution Verification

- Script executed successfully in the browser.
- Input was reflected without encoding, confirming Reflected XSS vulnerability.

XSS (Cross Site Scripting) Output:





- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)**
- XSS (Stored)
- CSP Bypass
- JavaScript
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello

More Information

- [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

Analysis:

- User input is not properly sanitized or encoded before rendering.
- Malicious scripts can execute in victims' browsers, leading to session hijacking or phishing attacks.

4.3 Burp Suite Usage Summary

- **Intercept Mode:** Paused requests to analyze parameters.
- **HTTP History:** Viewed GET requests and server responses.
- **Passive Crawl:** Added site URLs to site map for further analysis.

⚡

Burp

Project

Intruder

Repeater

View

Help

Burp Suite Community Edition v2026.3.2-46522 (Early Adopter) - Temporary Project

—

□

×

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

🌐

>

⚙️

Organizer

Extensions

Discover

Tasks

New scan

New live task

⏸️ ⚙️ ?

Filter ▾

Search 🔍

1. Live passive crawl from Proxy (all t... ⏸️ ⋮)

Add links. Add item itself, same domain and URLs in suite scope.

Capturing ☒

🔒 Time to level up? Catch more bugs with Burp Suite Pro Find out more ✕

🔖 1. Live passive crawl from Proxy (all traffic) ⏸️ ⚙️ ?

Summary

📁 Items added to site map [View site map](#)

Host	Meth...	URL	Status c...	MIME type
No items to show				

Event log

All issues

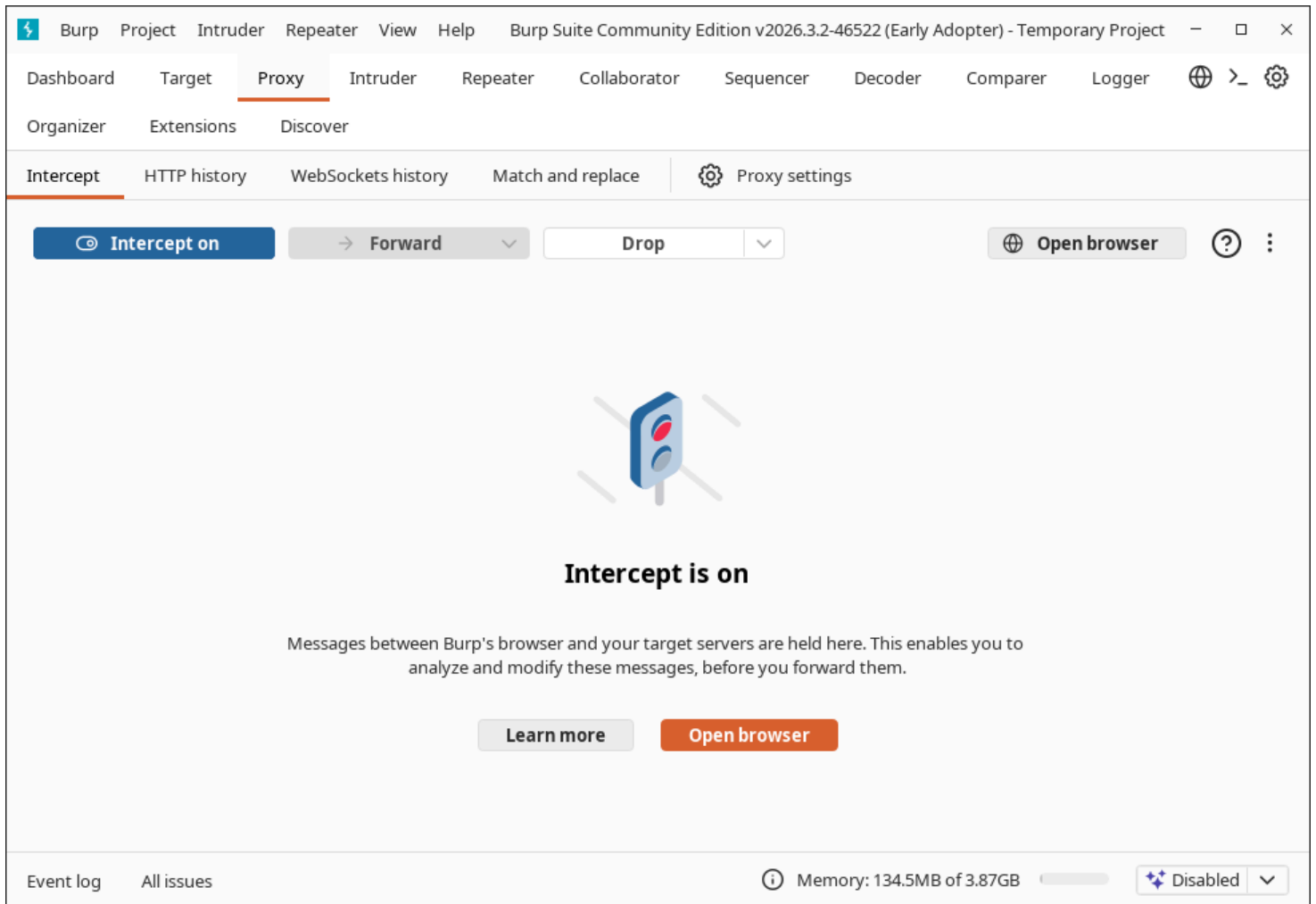
ⓘ Memory: 134.5MB of 3.87GB

Disabled ▾

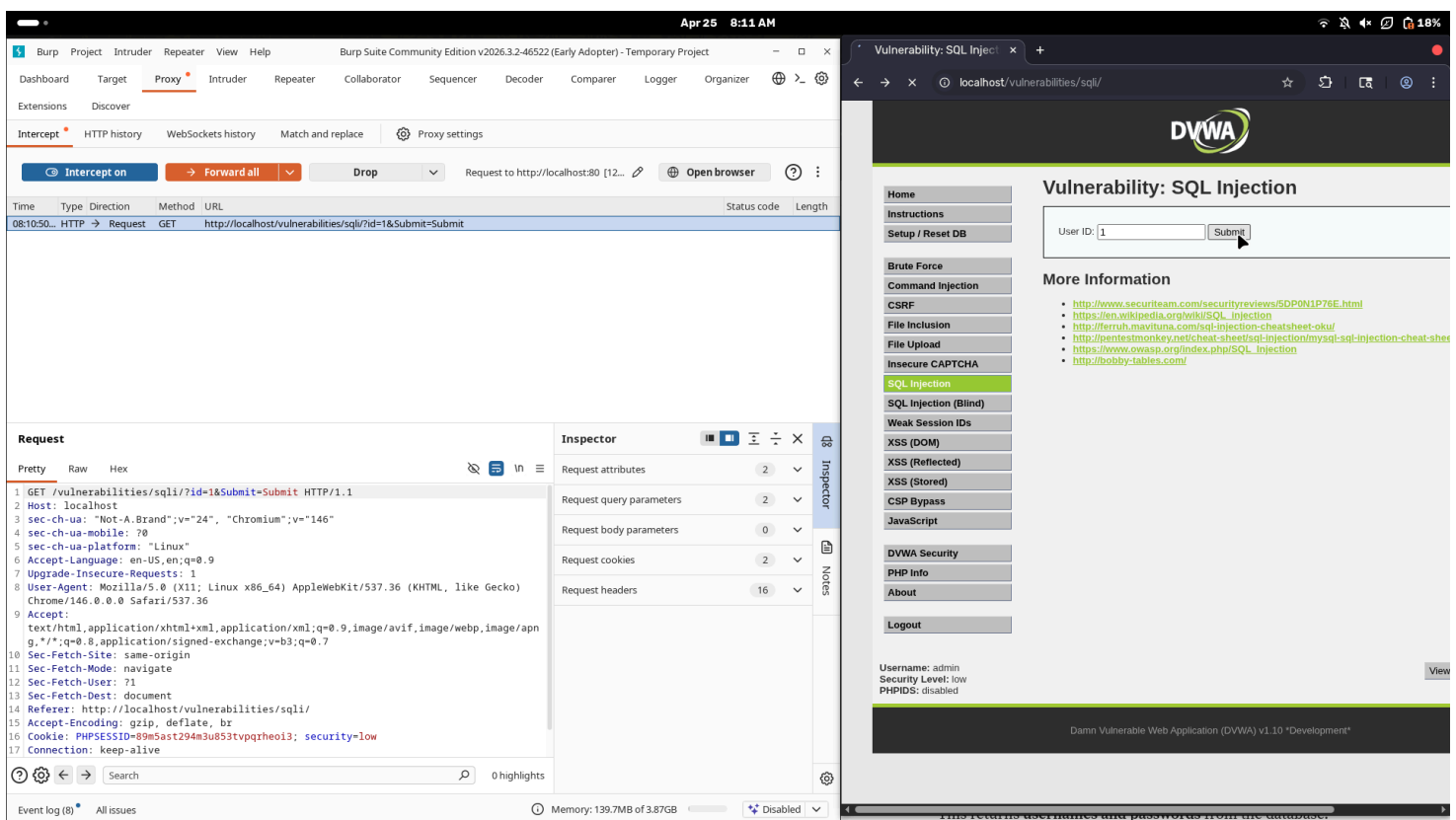
A screenshot of the Burp Suite Proxy tab. The top navigation bar includes 'Burp', 'Project', 'Intruder', 'Repeater', 'View', and 'Help'. Below this, a row of tool tabs shows 'Dashboard', 'Target', 'Proxy' (selected), 'Intruder', 'Repeater', 'Collaborator', 'Sequencer', 'Decoder', 'Comparer', and 'Logger'. A second row of tabs includes 'Organizer', 'Extensions', and 'Discover'. The main content area has a sub-header with 'Intercept' (selected), 'HTTP history', 'WebSockets history', 'Match and replace', and 'Proxy settings'. Below the sub-header, there's a control bar with 'Intercept off' (with an eye icon), a 'Forward' button with a dropdown arrow, a 'Drop' button with a dropdown arrow, an 'Open browser' button with a globe icon, and a help icon. The main display area features a large blue and green icon of a switch, the text 'Intercept is off', and a paragraph explaining that turning intercept on allows analyzing and modifying messages between the browser and target servers. At the bottom of this section are 'Learn more' and 'Open browser' buttons. The bottom status bar shows 'Event log', 'All issues', 'Memory: 134.5MB of 3.87GB', and a 'Disabled' status with a dropdown arrow.

We will switch to Burp's proxy tab to route all traffic through burp's proxy and intercept all the traffics.

Turn On the Intercept mode on Burp from the proxy tab



Intercept mode allowed pausing and modifying requests:



Insight: Burp Suite provided full visibility and control over HTTP requests, enabling precise testing and confirmation of vulnerabilities.

5. Mitigation Recommendations

5.1 SQL Injection

1. **Use Parameterized Queries / Prepared Statements**

```
SELECT * FROM users WHERE id = ?
```

- 2.
2. **Input Validation:** Only allow expected types (e.g., numeric for `id`).
3. **Error Handling:** Suppress detailed database error messages.
4. **Database Privileges:** Limit privileges of web application accounts.
5. **ORM Frameworks:** Avoid constructing raw SQL queries manually.

5.2 Cross-Site Scripting (XSS)

1. **Output Encoding:** Encode HTML special characters (`<` , `>` , `&`).
2. **Input Validation:** Restrict or sanitize input fields to reject scripts.
3. **Content Security Policy (CSP):** Enforce strict policies (`default-src 'self'`).
4. **Secure Frameworks:** Use frameworks that automatically escape outputs (React,Rust, GO, Python, Django, etc.).

5.3 General Recommendations

- Enforce HTTPS for all traffic.
- Deploy Web Application Firewall (WAF).
- Keep Libraries/Binaries & System Utilities Up-To-Date/Updated.

6. Conclusion

The DVWA application was tested under controlled conditions, demonstrating vulnerabilities to SQL Injection and Reflected XSS. Burp Suite proved effective in intercepting and manipulating requests to validate these security issues.

- SQL Injection allowed unauthorized access to all user records after refining the payload.
- Reflected XSS enabled script execution in the browser.
- Burpsuite allowed for intercepting & manipulation of requests.